Relatório de Desenvolvimento e Avaliação Experimental de um Servidor HTTP/1.1 com Suporte a QoS e Controle de Admissão em Linguagem C

Vanessa M. Otte[1], **Lorhan S. Lima**[1]

[1]Fundamentos e Avaliação de Redes de Computadores
Engenharia de Computação – Universidade Federal do Pampa (UNIPAMPA)
Bagé – RS – Brazil

{lorhanlima, vanessaotte}.aluno@unipampa.edu.br

Repositório GitHub: https://github.com/Vanessa-Otte/servidor-http-qos.git

Resumo. *Este relatório técnico apresenta a extensão arquitetural e a avaliação experimental do servidor Web baseado no protocolo HTTP/1.1, correspondente ao Produto Mínimo Viável 2 (MVP 2). Evoluindo a base multithread desenvolvida no MVP 1, foram incorporados mecanismos de Qualidade de Serviço (QoS), compreendendo controle de admissão baseado em capacidade global, modelagem de tráfego (traffic shaping) por IP e conexão, além de telemetria de RTT fim-a-fim em tempo real. A validação experimental foi conduzida em ambiente Linux virtualizado utilizando as ferramentas* `curl`*, IPTraf-ng,* `tcpdump` *e Wireshark. Os resultados obtidos comprovam a correta rejeição de excedentes via código* `503 Service Unavailable`*, a divisão equitativa de largura de banda entre conexões concorrentes e a estabilidade da taxa de transmissão imposta pelo servidor.*

1. Introdução

A evolução do protocolo HTTP/1.1 possibilitou o transporte eficiente de múltiplos recursos sobre uma mesma conexão persistente TCP, evitando a sobrecarga de sucessivos *Three-Way Handshakes*. Entretanto, servidores de alto desempenho demandam mecanismos ativos de controle para evitar a exaustão de largura de banda e impedir que requisições agressivas degradem o serviço prestado aos demais clientes da rede. Sem políticas explícitas de Qualidade de Serviço (QoS), fluxos concorrentes disputam o enlace de forma desordenada.

O objetivo desta etapa do projeto (MVP 2) consiste na concepção, implementação e avaliação de mecanismos de controle de tráfego e contenção de recursos em nível de aplicação, integrados à base concorrente desenvolvida em C com a biblioteca POSIX Threads (`pthreads`). O software passa a operar sob quatro pilares fundamentais:

1. **Controle de Admissão:** verificação estática da vazão global do servidor, rejeitando novas conexões com código `503 Service Unavailable` quando a capacidade configurada for extrapolada.

2. **Modelagem de Tráfego (*Traffic Shaping*):** regulação ativa da velocidade de envio de dados em blocos binários via temporização com `usleep()`, promovendo o compartilhamento justo de taxa entre fluxos simultâneos de um mesmo IP.
3. **Telemetria Fim-a-Fim:** estimativa em tempo de execução do tempo de ida e volta (RTT) entre a entrega da página HTML base e o primeiro recurso embutido requisitado pelo cliente.
4. **Painel Operacional em Tempo Real:** *thread* independente para renderização contínua das métricas de vazão alocada, conexões ativas por endereço e eventos de telemetria.

Este relatório documenta a arquitetura de software implementada no MVP 2 e apresenta a avaliação experimental realizada sobre a pilha de protocolos.

2. Desenvolvimento e Arquitetura do Software

A arquitetura do servidor preserva o modelo concorrente multithread baseado na interface de Sockets Berkeley, estendendo o processo para gerenciar o estado global de QoS sincronizado por exclusão mútua (`pthread_mutex_t`).

2.1. Estrutura de QoS e Controle de Admissão

As regras de controle de banda são carregadas na inicialização a partir do arquivo `config/taxas.txt`, associando endereços IP a taxas nominais em kbps. A capacidade máxima do servidor é informada diretamente via argumento de linha de comando.

A representação do estado global e dos clientes é gerenciada pela estrutura `EstadoQoS`:

```
typedef struct {
    char ip[INET_ADDRSTRLEN];
    double taxa_base_kbps;
    int conexoes_ativas;
} RegraIP;

typedef struct {
    RegraIP regras[MAX_CLIENTES];
    int qtd_regras;
    double vazao_maxima_servidor_kbps;
    double vazao_alocada_atual_kbps;
    char ultimo_log_rtt[128];
    pthread_mutex_t mutex;
} EstadoQoS;
```

O controle de admissão atua logo após a chamada `accept()`, antes do processamento dos cabeçalhos HTTP. A rotina verifica se a inclusão da taxa do cliente ultrapassará o teto estipulado para o servidor. Se excedido, uma resposta de rejeição é despachada e a conexão é encerrada:

```
pthread_mutex_lock(&g_qos.mutex);
if (g_qos.vazao_alocada_atual_kbps + taxa_cliente > g_qos.
    vazao_maxima_servidor_kbps) {
```

```
    pthread_mutex_unlock(&g_qos.mutex);
    char *resp503 = "HTTP/1.1 503 Service Unavailable\r\n"
                    "Content-Length: 35\r\n"
                    "Connection: close\r\n\r\n"
                    "<h1>503 - Capacidade Excedida</h1>";
    send(idCliente, resp503, strlen(resp503), 0);
    close(idCliente);
    free(ponteiroID);
    return NULL;
}
g_qos.vazao_alocada_atual_kbps += taxa_cliente;
pthread_mutex_unlock(&g_qos.mutex);
```

2.2. Modelagem de Tráfego (Traffic Shaping) e Estimativa de RTT

A regulação da vazão é executada durante a transmissão de arquivos binários (imagens). Quando um mesmo IP mantém múltiplas conexões simultâneas abertas, a taxa base configurada é dividida uniformemente entre seus fluxos:

$$\text{Taxa Efetiva} = \frac{\text{Taxa Base do IP}}{\text{Conexões Ativas}} \tag{1}$$

Para manter o fluxo contínuo sem estourar buffers de rede, a taxa nominal é convertida para bytes por segundo e o intervalo de espera entre blocos de 4096 bytes é calculado dinamicamente:

```
double taxa_bytes_seg = (taxa_efetiva_kbps * 1000.0) / 8.0;
double tempo_por_bloco = (double)bytesLidosDisco / taxa_bytes_seg;
useconds_t atraso_us = (useconds_t)(tempo_por_bloco * 1000000.0);

send(idCliente, bufferEnvio, bytesLidosDisco, 0);
if (atraso_us > 0) {
    usleep(atraso_us);
}
```

A estimativa do RTT fim-a-fim da camada de aplicação é obtida pela diferença temporal entre a entrega do documento base HTML (`t_html_req`) e a primeira requisição subsequente de um recurso embutido, disparada automaticamente pelo navegador sob a mesma conexão persistente:

```
if (eh_html) {
    t_html_req = agora;
} else if (!mediu_rtt && t_html_req.tv_sec > 0) {
    double delta_ms = (agora.tv_sec - t_html_req.tv_sec) * 1000.0 +
                      (agora.tv_usec - t_html_req.tv_usec) / 1000.0;
    mediu_rtt = 1;
    pthread_mutex_lock(&g_qos.mutex);
    snprintf(g_qos.ultimo_log_rtt, sizeof(g_qos.ultimo_log_rtt),
            "IP %s -> RTT Fim-a-Fim: %.2f ms", ipCliente, delta_ms);
```

```
    pthread_mutex_unlock(&g_qos.mutex);
}
```

2.3. Tratamento de Sinais e Painel de Monitoramento

Para evitar que o encerramento prematuro de transferências pelo cliente derrube o processo servidor via sinal `SIGPIPE`, foi realizado o tratamento explícito no início da rotina `main()`:

```
int main(int argc, char *argv[]) {
    signal(SIGPIPE, SIG_IGN);
    ...
```

Uma thread independente executa a função `thread_painel()`, realizando varreduras periódicas a cada 1 segundo para renderizar no terminal o estado do servidor com códigos de escape ANSI.

3. Avaliação Experimental

A validação prática do servidor foi conduzida no ambiente Linux virtualizado (WSL/Ubuntu). Foi estruturada uma página de teste (`galeria.html`) contendo seis imagens em alta resolução de múltiplos megabytes cada. Os testes empregaram ferramentas de injeção de requisições concorrentes (`curl`), analisadores de fluxo de baixo nível (IPTraf-ng) e coletores de pacotes (`tcpdump` e Wireshark) na interface de loopback (`lo`).

3.1. Validação do Controle de Admissão (Erro 503)

Para testar a limitação de capacidade, o servidor foi iniciado com o teto de 500 kbps, enquanto a regra do cliente `127.0.0.1` exigia uma alocação nominal de 4200 kbps.

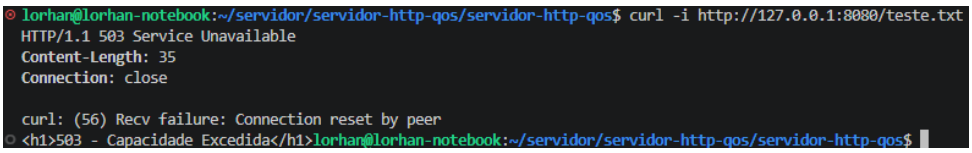


Figura 1. Resposta do servidor rejeitando a conexão com código de status HTTP/1.1 503 Service Unavailable devido à insuficiência de capacidade de vazão.

Como demonstrado na Figura 1, a requisição disparada pelo `curl` foi barrada com o cabeçalho `HTTP/1.1 503 Service Unavailable` e a diretiva `Connection: close`. O mecanismo garante que a saturação por novas demandas não comprometa as conexões já estabelecidas anteriormente.

3.2. Conexões Simultâneas e Compartilhamento de Banda no Mesmo IP

Avaliou-se a divisão equitativa de banda disparando dois processos paralelos do utilitário `curl` voltados para duas imagens repetidas, sob a taxa base configurada de 4200 kbps.

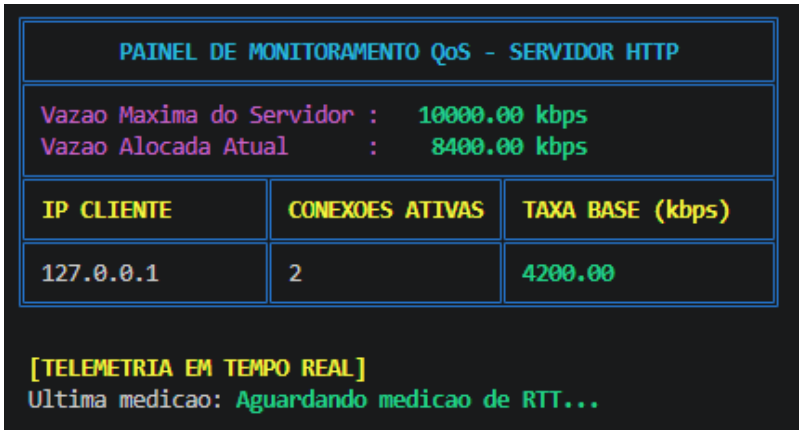


Figura 2. Painel de monitoramento registrando 2 conexões concorrentes ativas para o IP 127.0.0.1 e vazão agregada alocada de 8400.00 kbps.

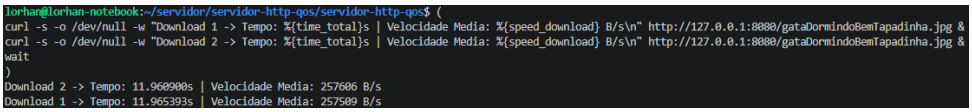


Figura 3. Taxas de transferência medidas simultaneamente pelo utilitário curl, convergindo para a divisão exata da largura de banda configurada.

O painel na Figura 2 confirma o rastreamento das 2 conexões simultâneas pelo mesmo host. Na Figura 3, a velocidade média obtida por cada fluxo estabilizou-se em aproximadamente 250 KB/s. Esse resultado comprova a divisão equitativa da taxa contratada (4200 kbps/2 = 2100 kbps $\approx$ 262.5 KB/s), sendo a sutil diferença explicada pela sobrecarga de cabeçalhos das camadas de transporte e aplicação.

3.3. Atendimento Concorrente a Clientes com Taxas Distintas

Para validar o suporte a clientes heterogêneos, foram mapeados dois endereços de loopback distintos no arquivo de configuração: `127.0.0.1` com taxa de 420 kbps e `127.0.0.2` com taxa de 1000 kbps.



Figura 4. Monitoramento dinâmico identificando fluxos simultâneos de diferentes hosts com taxas assimétricas configuradas.

A Figura 4 evidencia a correta segregação de parâmetros por cliente. O

servidor alocou os recursos de forma independente, somando exatamente 1420.00 kbps de vazão alocada sob a capacidade total de 20000.00 kbps.

3.4. Monitoramento com IPTraf-ng e Captura com TCPdump

O comportamento temporal do fluxo binário e o perfil de tráfego foram observados em baixo nível com o IPTraf-ng. Configurou-se a taxa de 420 kbps para o download de uma imagem de 2.93 MB.

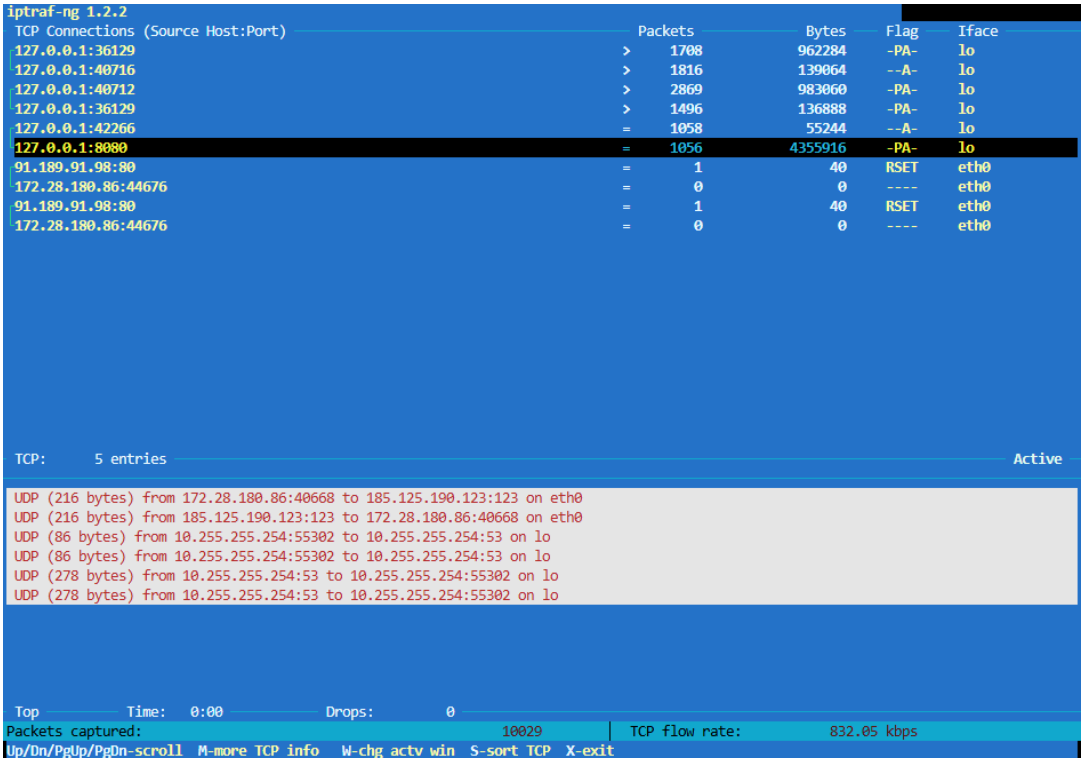


Figura 5. Monitoramento via IPTraf-ng e curl evidenciando a cadência estável da vazão sob modulação de 420 kbps.

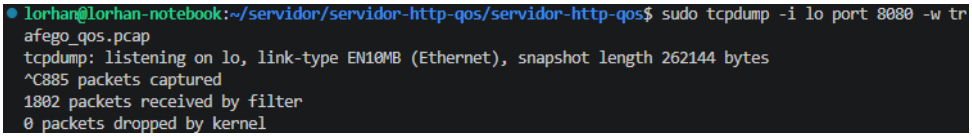


Figura 6. Registro de execução do utilitário tcpdump indicando a captura de 885 pacotes sem descarte pelo kernel.

Como observado na Figura 5, a velocidade permaneceu fixada em 52.5 KB/s (420.000/8 = 52.500 Bytes/s), atestando a linearidade do envio. Ao mesmo tempo, o tráfego foi registrado via comando `sudo tcpdump -i lo port 8080 -w trafego_qos.pcap`, resultando na captura íntegra de 885 pacotes (Figura 6).

3.5. Análise Gráfica de Vazão no Wireshark (I/O Graphs)

O arquivo de captura `trafego_qos.pcap` gerado pelo `tcpdump` foi analisado na ferramenta Wireshark utilizando a funcionalidade *Statistics → I/O Graphs*.

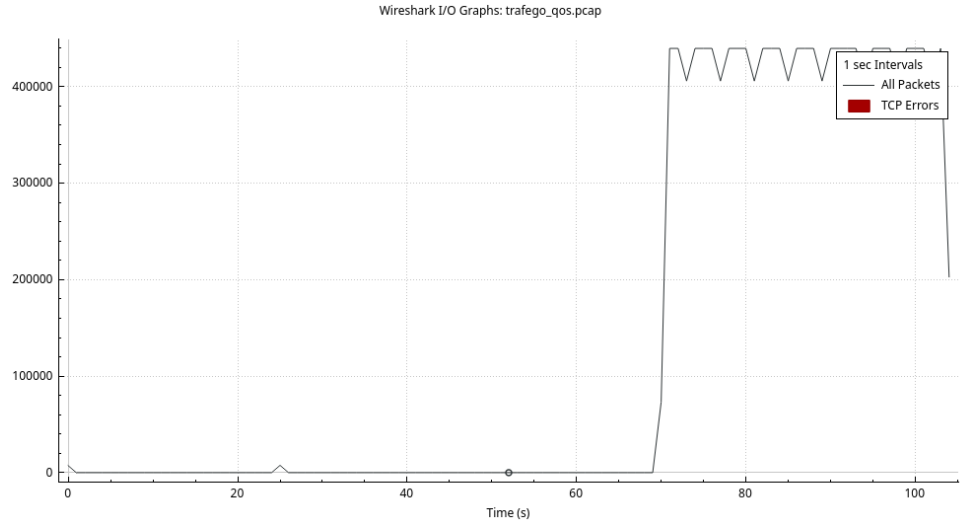


Figura 7. Gráfico de I/O no Wireshark demonstrando o patamar estável de transmissão em torno de 420.000 bits/s.

A Figura 7 demosntra o comportamento do *traffic shaping*. A curva de tráfego cresce logo após o handshake e atinge um pico rigorosamente plano em 420.000 bits/s, validando a eficácia do algoritmo de atraso artificial na contenção de rajadas.

3.6. Telemetria de RTT em Tempo Real

A estimativa do RTT fim-a-fim foi avaliada durante o carregamento completo da galeria via navegador Web, em requisições consecutivas persistentes.

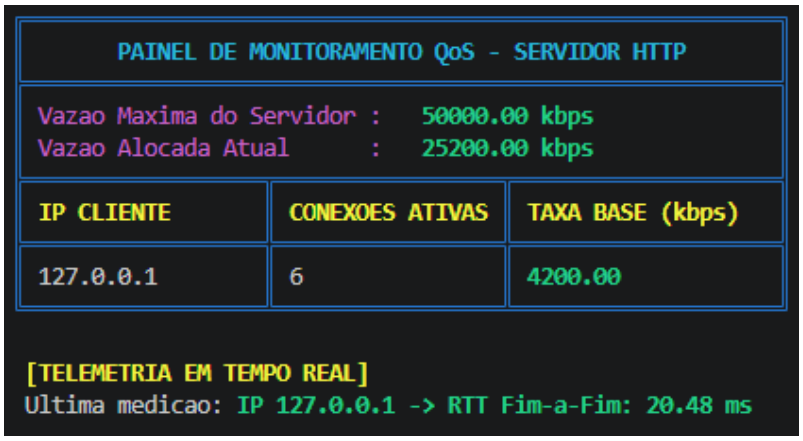


Figura 8. Painel de telemetria registrando RTT de 20.48 ms durante requisições subsequentes do navegador.

Conforme ilustrado na Figura 8, o servidor registrou com sucesso a métrica de 20.48 ms entre a solicitação inicial do documento HTML e a chegada do pedido da primeira imagem embutida, atualizando a seção de telemetria do painel dinâmico.

4. Conclusões

A implementação do Produto Mínimo Viável 2 (MVP 2) cumpriu com sucesso os requisitos experimentais e arquiteturais de Qualidade de Serviço para o servidor

HTTP/1.1. O controle de admissão mostrou-se estrito ao rejeitar conexões que excedessem a capacidade do sistema, enquanto o mecanismo de temporização viabilizou uma modelagem de tráfego estável e justa entre fluxos concorrentes. As medições convergentes obtidas por meio de `curl`, IPTraf-ng e gráficos de I/O do Wireshark comprovaram empiricamente o controle de taxa pretendido. A thread de monitoramento garantiu a visibilidade contínua do estado do servidor sem corrupção de saída por condições de corrida.